High-level approach for generating household data, dialogs, and extraction metrics

This page summarizes the end-to-end methodology across data generation, dialog generation, extraction scoring, Train/holdout test split construction, and control benchmark results.

Household scenarios	Dialogs generated	Extraction metrics scored	Overall extraction accuracy
5,000	316	99	98.74%
10 scenarios × 500 households	300 single-turn + 16 multi-turn	7 metrics × 3 models × 3 splits	Across all metrics and splits, including control benchmarks

Overview · Data generation · Dialog generation · Metrics · Train and holdout split · Results · Quality control · Limitations

Overview

The benchmark is built as a controlled synthetic-data pipeline

1 Generate Households
Sample reference US ZIPs, assign household profiles, and create synthetic people with realistic attributes, relations, and ages.

2 Generate dialogs
Create long utterance-level transcripts from each household profile, with evidence for the field-level extraction targets.

3 Extract and score
Run structured extraction from transcripts and compare predictions against ground truth across field-level metrics.

4 Validate holdout split
Check that the held-out set is not leaking from the training distribution in structure or evidence.

Table of contents

1. Household-based generation
2. Scenario-conditioned dialog generation
3. Blind extraction, scoring, and ablations
4. Train and holdout split construction
5. Train and holdout test results
6. Frozen control set stress tests
7. Limitations

5,000 synthetic data is scenario-driven and distributionally checked

10 scenarios
Each scenario has 500 households in the current artifacts. The model fields vary across 7 under-5-coverage subfields.

85k+ utterances
Dialogs average 85.8 utterances in the current artifacts. The model fields vary across 7 under-5-coverage subfields.

85k+ extraction targets
Across all splits, the current artifacts contain 85,800 extraction targets.

0 schema violations
Current artifacts show no schema violations in the current artifacts.

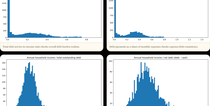
The generator creates offline US-scale household rosters that provide geographic variety, demographic plausibility, relatives, and age-consistent relationships. Each household then conditions dialog generation, extraction, and evaluation.

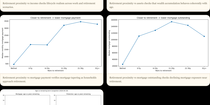
5,000 synthetic households

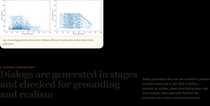
Scenario	Count
healthy_public_earlyage_adult_children	498
affluent_couple_2-order-age_adult_parents	741
rural_extended_adult_child	1,121
one_child_nuclear_young_parents	512
young_adult_roommates_low_income	1,111
single_parent_child_low_income	497
multi_ethnic_urban_mixed_household	498	8.43%
financially_stressed_with_dependents	491	8.45%
retirement_couple_no_dependents_income	498	8.43%
accessibility_needed_paying_caregiver	257	5.14%
disabled	1,121	22.42%
children	2,379	47.58%
adult	2,621	52.42%
male	2,500	50.00%
female	2,500	50.00%
children	2,379	3.09%

Selected data-generation diagnostics

The following figures are a representative slice of the diagnostic plots produced by Stage 1c. Together they illustrate how we check scenario coverage, marginal distributions, cross-field dependencies, and household-level structure before dialog generation.







Monthly mortgage payment
Monthly mortgage payment
Count
Monthly mortgage pay

Gemini, Claude, and others. Ask each model for a concrete action plan for improving realism, aggregate feedback across dialogs and models, and iteratively update the dialog-generation prompts.

Examples include typos, transcript artifacts, repeated turns, missing punctuation, speaker-label noise, irrelevant side conversations, partial truncation, and contradictory corrections.

Use these attacks to design defenses, stronger validation, or a more robust extraction architecture.

4. Compare structured output modes

Compare extraction with and without structured generation / structured output constraints. Measure not only accuracy, but also parse failures, coercion issues, latency, retry rate, and cost.

5. Study dialog length versus extraction quality

Quantify how transcript length affects extraction quality and latency. Identify the dialog size at which extraction should switch from single-pass processing to batching, chunking, or hierarchical extraction.

6. Add a pre-enrichment mode

Improve client experience by pre-filling as much information as possible from public or permissioned sources, such as public professional profiles or other available records. The dialog can then focus only on missing, ambiguous, or stale facts, shortening the conversation while still refreshing outdated information.

7. Make infrastructure failure-tolerant

Design graceful failover for API outages and provider degradation. For example, if GPT is unavailable, switch to Gemini or another provider, but only through a pipeline that has been explicitly tested and calibrated for that fallback model.

8. Evaluate multiple extractor models and ensembles

Test different LLMs as extractors and compare single-model, fallback, voting, and specialist-ensemble setups. Track accuracy, field-specific failure modes, latency, cost, and operational reliability.

9. Run real-world A/B validation

Compare this pipeline against the current production approach on real dialogs. Measure actual error patterns, human review burden, correction rates, and how extraction errors correlate with downstream product metrics.

ARTIFACT LINKS

Useful local references

These links point to files in this repository and should work when opening this HTML from the repo root.

Data generation

- 01 approach
- 01 README
- Scenario coverage CSV
- Scenario coverage plot

Dialogs

- 02 README
- DeepSeek prompt
- Train grounded GT
- Holdout test registry

Extraction and results

- 03 approach
- 03 README
- Train metrics
- Holdout test metrics
- Train discrepancy report
- Holdout test discrepancy report